

CCF2026开源创新大赛

Mooncake 赛题2初赛技术报告

Mooncake Store 碎片感知分配优化

Fragmentation-Aware Allocation for Mooncake Store

参赛方向	赛题2：优化Mooncake Store吞吐性能、高可用功能和可扩展性，优化SGLang HiCache+Mooncake Store 性能
作品名称	Mooncake Store 碎片感知分配策略
参赛仓库	https://github.com/Lorry1024/ccf2026-mooncake-fragmentation-aware
上游PR	https://github.com/kvcache-ai/Mooncake/pull/2797
PR头提交	0123fa1 Fix fragmentation-aware allocation test setup
报告日期	2026年7月9日

本报告与仓库中的patch、复现实验、验证日志和展示PPT共同构成初赛提交材料。

摘要

Mooncake Store 是 Mooncake 体系中支撑 KVCache 对象存储和复用的关键组件。现有 `free_ratio_first` 策略根据 `segment` 总空闲比例进行排序，能够改善整体空间利用均衡，但在混合大小 KVCache 对象和长时间运行的场景下，总空闲空间可能被切分为多个小空洞，无法真实反映当前大对象是否可以被连续放入。本文提出并实现 `fragmentation_aware` 策略，在保持默认行为不变和有界候选采样不变的前提下，将最大连续空闲区域、连续性比例和当前请求可容纳性纳入排序逻辑，使 Master 优先选择能够直接满足请求的 `segment`。

本作品已形成 Mooncake 上游 draft PR，当前 PR 头提交 `0123fa1` 已通过 GitHub Actions，结果为 26 个检查成功、1 个检查跳过。确定性专题仿真显示，在 6 个大对象请求上，`free_ratio_first` 的首选 `segment` 可直接容纳次数为 0/6，而 `fragmentation_aware` 为 6/6；`fallback` 尝试次数从 11 降至 0；平均候选 `segment` 数量保持 5.00；本地单次决策耗时从 121.00ns 增加到 138.93ns，额外开销约 17.93ns。该结果说明，新策略能够以很小的决策成本提升碎片化场景下的分配稳定性。

关键词： Mooncake Store；KVCache；碎片感知；连续空闲区域；分配策略；SGLang HiCache

模块完成情况

表 1: 初赛阶段模块完成情况

模块	完成情况
问题分析	明确 <code>free_ratio_first</code> 在碎片化大对象分配中的误判模式，形成最小复现和专题仿真。
策略设计	设计 <code>fragmentation_aware</code> 排序规则，优先考虑 <code>can_fit</code> 、连续性评分和最大连续空闲区域。
代码实现	在 Mooncake Store 中新增策略类型、策略类、配置解析、启动参数和 <code>benchmark</code> 矩阵。
测试验证	新增碎片化单元测试，保留 <code>preferred segment</code> 语义，PR 通过上游 GitHub Actions。
材料整理	完成中文 Markdown 材料、LaTeX 技术报告、展示 PPT、复现实验源码和提交包。
待完成项	5 分钟展示视频尚未录制；真实 RDMA 和 SGLang HiCache 端到端 <code>benchmark</code> 作为后续工作。

目录

1	赛题理解与项目定位	1
1.1	技术路线总览	1
2	原有机制与问题分析	2
2.1	Mooncake Store 分配链路 with 关键抽象	2
2.2	KVCache 对象生命周期 with 碎片来源	3
2.3	现有策略的能力边界	4
2.4	总空闲比例误判的形成机制	5
2.5	误判带来的性能影响	5
2.6	为什么不能只依赖 fallback	5
2.7	本项目抽象出的优化需求	7
3	碎片感知分配策略设计	7
3.1	设计目标 with 非目标	7
3.2	策略输入、输出 with 不变量	8
3.3	候选元数据 with 评分字段	8
3.4	排序规则	9
3.5	权重选择的工程解释	10
3.6	候选排序示例	10
3.7	完整策略流程	11
3.8	多副本场景下的处理	12
3.9	退化行为 with 边界保护	12
3.10	复杂度 with 并发安全分析	13
4	实现路径	13
4.1	代码改动总览	14
4.2	策略类接入	14
4.3	核心执行流程	14
4.4	配置和工厂接入	17
4.5	兼容性、灰度 with 回滚	18
4.6	单元测试设计	18

4.7	测试修复与CI闭环	19
4.8	文档、benchmark与复现材料	19
5	验证与效果	21
5.1	验证思路	21
5.2	CI验证	21
5.3	专题仿真指标	22
5.4	结果解释	22
6	与赛题要求的对应	23
7	使用与复现	24
7.1	启用策略	24
7.2	应用补丁	24
7.3	回滚补丁	24
7.4	复现实验	24
8	边界与后续工作	25
9	结论	25
A	提交材料清单	25
B	参考链接	26

1 赛题理解与项目定位

Mooncake 赛题2 要求围绕 Mooncake Store 吞吐性能、高可用功能、可扩展性以及 SGLang HiCache+Mooncake Store 性能开展优化。该要求并不只局限于网络传输或外层推理框架，也包含 Store 后端内部影响吞吐稳定性和资源利用率的关键路径。对 KV-Cache 场景而言，Store 中对象的分配位置会影响一次写入是否能顺利落地，也会影响后续 fallback、重试、淘汰和副本放置压力。

本项目选择的切入点是 Mooncake Store 分配策略层。该方向具有三个特点：第一，它位于真实 Mooncake Store 代码路径中，能够通过 PR 形式提交给上游；第二，它对 SGLang HiCache 等前端具有后端相关性，因为这些前端将 KVCache 对象交给 Store 时需要依赖 Store 的放置决策；第三，它可以在不重写协议、不引入重依赖、不改变默认行为的条件下完成可验证优化。

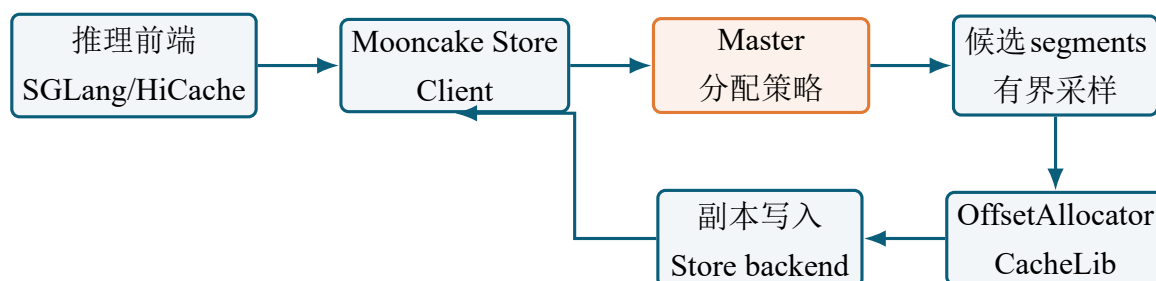


图 1: 本项目在 Mooncake Store 路径中的位置

1.1 技术路线总览

本项目的技术路线可以概括为“先把碎片化问题显式建模，再把模型压缩为可在现有分配路径中低成本执行的排序信号”。我们没有从修改网络协议、重写存储后端或引入全局整理线程入手，原因是这些方向虽然可能带来更大范围的能力提升，但会显著增加验证复杂度，并且容易影响 Mooncake Store 已有部署行为。赛题初赛更需要一个边界清晰、证据充分、能够被上游项目评审的优化点。因此，本项目把关注点收敛到 Master 侧 segment 选择过程：当 Store 面对一个已知大小的 KVCache 对象时，应该优先选择“当前真的能放下”的 segment，而不是只选择“总空闲比例看起来更高”的 segment。

从工程路径看，完整工作被拆成六步。第一步，阅读 Mooncake Store 已有分配策略，确认 random、free_ratio_first 等策略的职责边界。第二步，构造最小碎片化反例，证明总空闲比例和连续可用空间之间存在可观察的不一致。第三步，设计 fragmentation_aware 评分字段，把最大连续空闲区域、连续性比例和当前请求大小引入候选排序。第四步，把策略接入 Mooncake Store 正式配置路径，而不是停留在离线脚本或实验代码中。第五步，补充单元测试、benchmark 矩阵和文档说明，保证

策略可使用、可回滚、可被评审。第六步，通过上游PR和GitHub Actions完成开源项目层面的基本验证。

表 2: 从赛题要求到本项目技术路线的分解

层次	具体问题	本项目处理方式
赛题目标	提升Mooncake Store吞吐、可扩展性和HiCache后端表现	选择Store内部高频放置决策作为优化入口
系统现象	混合大小对象使segment内部出现碎片	以最大连续空闲区域刻画碎片化对当前请求的影响
原有不足	<code>free_ratio_first</code> 只根据总空闲比例排序	增加 <code>can_fit</code> 和连续性评分，使排序更贴近真实可分配性
工程约束	不能破坏默认行为、协议和副本语义	作为显式可选策略接入，保留 <code>preferred</code> 、 <code>excluded</code> 和 <code>fallback</code> 路径
验证方式	初赛无法完全覆盖真实RDMA和SGLang集群	先用单元测试、专题仿真、 <code>benchmark</code> 接入和上游CI证明策略正确性

2 原有机制与问题分析

2.1 Mooncake Store分配链路与关键抽象

Mooncake Store在写入KVCache对象时，需要为对象的每个副本选择可写入的segment。上层调用并不直接关心底层空闲块的形态，而是把对象大小、副本数、`preferred segment`和`excluded segment`等约束交给Master侧分配逻辑。Master再调用具体分配策略，从可用segment集合中选出候选，并通过对应allocator完成空间申请。这个过程看似只是“选择一个segment”，但实际承担了三类语义：

1. 容量语义。当前segment是否拥有足够空间，决定对象是否可以一次性落地。
2. 副本语义。多副本写入需要避免重复使用同一segment，并在候选不足时保留`best-effort fallback`。
3. 调度语义。`preferred segment`表示上层希望优先使用的放置位置，`excluded segment`表示本次请求不能选择的位置。

因此，分配策略不是孤立的性能小优化，而是Store吞吐稳定性、资源利用率和高可用副本放置之间的连接点。如果策略选中了“总空闲很多但无法容纳当前对象”的segment，后续allocator仍会拒绝分配，系统只能继续尝试其他segment或走fallback路径。该失败不会直接制造数据错误，但会增加一次无效尝试，并可能放大为锁竞争、重试、尾延迟和副本放置压力。

表 3: Mooncake Store 分配路径中的关键对象

对象	主要职责	与本项目的关系
Master	管理segment视图并执行分配策略	新策略接入点位于Master侧策略工厂和配置解析路径
segment	Store后端的空间放置单元	每个segment的空闲空间形态决定当前对象是否能落地
allocator	维护segment内部可用空间	提供总空闲空间和最大连续空闲区域等统计信息
preferred segments	上层传入的优先候选位置	新策略必须先尝试preferred，不能因为评分逻辑改变用户语义
excluded segments	当前请求禁止选择的位置	新策略在采样和fallback阶段都必须尊重排除集合

2.2 KVCache对象生命周期与碎片来源

KVCache对象的存储行为和普通定长块写入不同。推理系统中不同请求的上下文长度、batch形态、prefix共享程度和缓存淘汰时机都可能不同，导致写入Store的对象大小呈现明显差异。一个较短请求可能只产生较小的KVCache对象，一个长上下文请求可能产生更大的对象；多轮对话或prefix复用又会让对象生命周期不完全同步。当这些对象交错写入、交错释放时，同一个segment内部就会逐渐形成不规则的空闲区域。

可以把segment想象成一个连续地址空间。初始状态下，segment中空闲区域通常比较完整，任何大小合理的对象都容易落入。运行一段时间后，不同大小对象被插入其中；再运行一段时间后，部分对象被释放，释放位置不一定相邻。此时虽然空闲字节总量可能很多，但这些字节被分散在多个小区间中。对allocator而言，当前请求是否可分配，取决于是否存在一个足够大的连续空闲区间，而不是所有小区间相加是否足够。

这一点正是赛题2中“Store吞吐性能”和“SGLang HiCache+Mooncake Store性能”之间的连接处。HiCache等前端把KVCache对象交给Store，并期望后端尽快完成存放和复用。如果Store在碎片化场景下反复优先选择不可容纳的大对象位置，就会增加后

表 4: KVCache Store 中碎片形成的典型原因

来源	具体表现	对分配策略的影响
对象大小混合	不同请求产生的 KVCache 对象长度差异较大	大对象更依赖连续空闲区域，小对象更容易填入零散空洞
生命周期不同步	部分缓存被提前淘汰，部分缓存长期保留	空闲区间分散，segment 内部形态随时间恶化
多副本写入	同一对象可能需要放入多个 segment	一个副本的错误首选会放大整体 fallback 次数
前缀复用与淘汰	复用命中和淘汰策略改变对象保留时间	同批写入对象不一定同批释放，空间回收不规则
长时间运行	分配和释放持续交替发生	总空闲比例逐渐不能代表当前大对象可分配性

端分配路径的不确定性。即使最终能够通过 fallback 找到可用位置，前面失败的尝试仍然消耗了决策、锁、状态检查和日志路径资源。

2.3 现有策略的能力边界

Mooncake Store 已有多种分配策略。默认 random 策略开销低、行为简单，适合通用场景；free_ratio_first 策略按照总空闲比例排序，目标是让更空闲的 segment 优先吸收写入，从而改善整体空间利用均衡；ssd_free_ratio_first、cxl 和 local_first 等策略则面向特定介质或拓扑。就赛题2而言，free_ratio_first 是最重要的对比基线，因为它已经使用了空间信息，却仍然没有识别 segment 内部碎片。

free_ratio_first 的核心假设可以概括为：总空闲比例越高，当前请求越可能分配成功。该假设在对象大小比较均匀、释放行为比较规则、segment 内部空闲区较少被切碎时通常成立。但 KVCache Store 的运行状态并不总是满足这些条件。长上下文推理、不同 batch 形态、前缀复用和缓存淘汰会让对象大小差异扩大；对象生命周期不同步又会让释放位置分散。最终，一个 segment 可能拥有较高总空闲字节数，但这些空间被拆成多个小洞，无法提供一个足够大的连续区域。

2.4 总空闲比例误判的形成机制

在 allocator 需要连续空间的前提下，“总空闲空间”和“最大连续空闲区域”是两个不同指标。前者衡量还有多少空间未被使用，后者衡量当前请求能否被一次性放入。碎片化场景下，两者可能明显背离。图2展示了典型情况：上方 segment 的绿色空闲块数量多、总量不低，但每个块都较小；下方 segment 的总空闲量略低，却拥有足够大的连续区域。

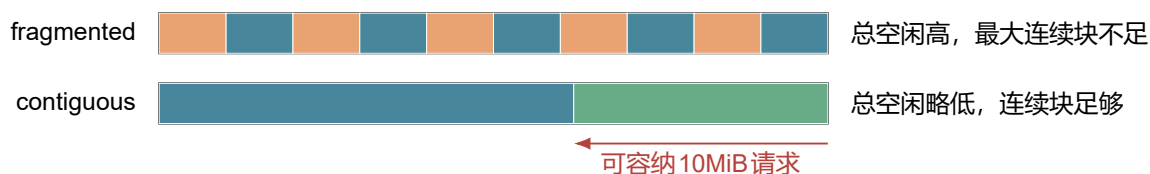


图 2: 总空闲空间与最大连续空闲区域可能不一致

最小复现数据如表5所示。`free_ratio_first` 会优先选择总空闲空间为 16MiB 的 `fragmented`，因为它的总空闲比例更高；但这个 segment 的最大连续空闲区域只有 8MiB，无法容纳 10MiB 请求。相反，`contiguous` 虽然总空闲空间只有 12MiB，却可以直接完成本次分配。

表 5: 最小碎片化复现场景

segment	总容量	总空闲	最大连续空闲	10MiB 请求
fragmented	32MiB	16MiB	8MiB	不能容纳
contiguous	32MiB	12MiB	12MiB	可以容纳

2.5 误判带来的性能影响

该问题的关键不在于最终一定失败，而在于首选候选质量下降。对多副本写入而言，一次错误排序可能导致一个副本先失败再尝试其他 segment；如果多个副本同时处于碎片化状态，失败尝试会被放大。对推理系统而言，这类失败尝试会影响 Store 写入路径的稳定性，进而影响 KVCache 对象的后端落盘、复用和淘汰压力。

2.6 为什么不能只依赖 fallback

一种直观疑问是：既然 `free_ratio_first` 选错后仍然可以继续尝试其他 segment，为什么还需要新增策略？原因在于 `fallback` 解决的是“最终能否找到位置”，而不是“首选路径是否稳定”。在高吞吐推理服务中，Store 面对的是连续不断的对象写入请求。一

表 6: 总空闲比例误判的影响链路

阶段	表现	对系统的影响
候选排序	高总空闲但碎片化的 segment 排在前面	首选 segment 质量下降
allocator 申请	最大连续区域小于请求大小	出现一次无效分配尝试
副本放置	当前副本继续尝试其他 segment	增加 fallback 和重试路径压力
长时间运行	释放和再分配持续制造小空洞	问题随对象大小混合程度和运行时长加重
推理场景	KVCache 对象写入路径不稳定	间接影响 HiCache 后端稳定性和尾延迟

次请求多一次失败尝试看似开销有限，但在高并发、多副本、大对象和长时间运行组合下，会形成累计影响。

首先，fallback 通常意味着前一次候选已经做过元数据读取、排除集合判断、allocator 尝试和失败处理。其次，多副本请求中每个副本都要避免重复使用同一 segment，因此一个副本的失败会改变后续副本的候选空间。再次，fallback 路径本质上是补救逻辑，它可以提高可用性，但不应成为碎片化场景下的常态路径。更理想的做法是在排序阶段就把明显不可容纳的候选排到后面，让系统优先尝试成功概率更高的 segment。

表 7: 依赖 fallback 与碎片感知排序的差异

比较项	依赖 fallback	碎片感知排序
决策位置	allocator 失败后补救	allocator 尝试前优化候选顺序
首选质量	可能被总空闲比例误导	优先判断最大连续空闲区域是否足够
多副本影响	每个副本都可能经历额外失败尝试	尽量让每个副本从高质量候选开始
延迟稳定性	失败次数随碎片状态波动	减少明显无效候选带来的尾部波动
工程语义	保底路径，不能删除	主路径排序优化，可与 fallback 共存

因此，本项目并不是替代 fallback，而是减少不必要 fallback。保留 fallback 是为了处理并发状态变化、候选不足、元数据估计不精确等不可避免情况；新增 fragmentation_aware 是为了让正常路径更少走到补救路径。这种关系更符合生产系统设计：主路径尽量准确，兜底路径必须存在，但不应成为常态。

2.7 本项目抽象出的优化需求

基于上述分析，本项目没有选择重写 Store 协议或引入复杂全局整理机制，而是把问题收敛为“在现有候选集合内提高排序质量”。这个抽象有明确边界：它不尝试移动已有对象，不做在线压缩，不改变副本协议，也不扩大候选扫描范围；它只利用 allocator 已经能提供或可近似得到的元数据，让策略在面对大对象请求时先识别“当前是否真的能放下”。因此，优化需求可以拆成四点：

1. 在排序中显式加入最大连续空闲区域，而不是只看总空闲比例。
2. 在当前请求大小已知的情况下，优先选择 `can_fit=true` 的候选。
3. 保持原有采样规模和 fallback 语义，避免策略本身成为新的扩展性瓶颈。
4. 保持配置可选和默认行为不变，便于灰度、回滚和上游接受。

3 碎片感知分配策略设计

3.1 设计目标与非目标

`fragmentation_aware` 的目标是提高碎片化场景下的首选 segment 可用性。它不是一个通用碎片整理器，也不是面向所有场景的强制默认策略。设计时刻意区分目标和非目标，是为了避免优化范围失控。

表 8: `fragmentation_aware` 的设计边界

类型	说明
目标1	在混合大小对象场景中，优先选择最大连续空闲区域能够容纳当前请求的 segment。
目标2	沿用 <code>free_ratio_first</code> 的有界候选思想，只改变候选排序信号，不扩大搜索范围。
目标3	保留 preferred segment、excluded segment、多副本去重和 best-effort fallback 语义。
目标4	作为显式可选策略接入，不改变 <code>random</code> 默认行为。
非目标1	不进行对象迁移、在线压缩或全局碎片整理。
非目标2	不修改 Mooncake Store 客户端协议、metadata 协议或后端存储格式。
非目标3	不宣称替代真实 RDMA 集群和 SGLang HiCache 端到端压测。

3.2 策略输入、输出与不变量

`fragmentation_aware` 并不是独立调度器，而是 Mooncake Store 分配策略体系中的一个可选实现。因此，它的输入和输出必须严格服从已有接口约束。策略输入主要包括当前请求的 `slice` 大小、副本数量、`allocator manager` 中的 `segment` 视图、`preferred segment` 列表和 `excluded segment` 集合。策略输出不是全局放置计划，而是一组已成功申请的 `buffer` 句柄；每个成功结果都对应一个实际 `allocator` 申请。换言之，评分阶段只决定尝试顺序，真正的空间占用仍以 `allocator` 返回结果为准。

本项目在设计中保留三个不变量。第一，任何被 `excluded` 的 `segment` 都不能被选中。第二，同一次多副本分配中，已经成功使用的 `segment` 会进入 `used` 集合，后续副本不能重复选择。第三，`preferred segment` 具有显式优先级，只要 `preferred` 可用并能完成分配，就不进入后续评分排序。这些不变量保证新策略不会改变调用方已经依赖的放置语义。

表 9: `fragmentation_aware` 的输入、输出与保持的不变量

类型	内容	约束说明
输入	请求大小 <code>slice_length</code>	用于判断最大连续空闲区域是否能够容纳当前对象
输入	副本数 <code>replica_num</code>	决定需要返回多少个成功分配结果
输入	<code>preferred segments</code>	先于采样候选尝试，保留上层显式调度语义
输入	<code>excluded segments</code>	在 <code>preferred</code> 、采样和 <code>fallback</code> 阶段均不得选择
内部状态	<code>used segments</code>	避免同一次多副本请求重复使用同一 <code>segment</code>
输出	成功申请的 <code>buffer</code> 列表	每个结果都来自 <code>allocator</code> 实际分配，而不是评分阶段的预测

3.3 候选元数据与评分字段

策略对每个采样候选 `segment` 计算五类字段。这里的关键是把“容量总量”和“空间形态”拆开：`free_ratio` 继续描述整体空闲程度，`largest_free_region` 和 `contiguity_ratio` 描述空闲区域是否集中，`can_fit` 则把当前请求大小纳入判断。

在实现中，若某类 `allocator` 无法提供精确的最大连续空闲区域，策略退化为使用可用空闲空间作为估计值。这样做的原因是兼容性优先：新策略不能要求所有 `allocator` 一次性改造完成。对能提供精确碎片信息的 `OffsetAllocator`，策略会使用真实

表 10: 候选segment评分字段

字段	计算方式	设计含义
total_free	所有 allocator 可用空间之和	描述 segment 剩余空间总量
largest_free_region	allocator 报告的最大连续空闲区域最大值	描述当前对象是否存在可落入的连续块
free_ratio	$\text{total_free} / \text{total_capacity}$	保留原有空间均衡信号
contiguity_ratio	$\text{largest_free_region} / \text{total_free}$	衡量空闲空间是否集中
can_fit	$\text{largest_free_region} \geq \text{request_size}$	直接判断当前请求是否可被候选容纳

`getLargestFreeRegion()`; 对无法提供精确值的场景，行为退化为接近总空闲视角，不会阻断已有部署。

3.4 排序规则

评分公式如下：

$$\text{free_ratio} = \frac{\text{total_free}}{\text{total_capacity}} \quad (1)$$

$$\text{contiguity_ratio} = \frac{\text{largest_free_region}}{\text{total_free}} \quad (2)$$

$$\text{score} = 0.70 \times \text{contiguity_ratio} + 0.30 \times \text{free_ratio} \quad (3)$$

其中 0.70 和 0.30 的权重体现了本策略的取舍：在碎片化优化场景下，空闲空间是否集中比总空闲比例更关键；但完全忽略总空闲比例也不合理，因为当多个候选都能容纳当前请求时，更高总空闲比例有助于维持 segment 之间的负载均衡。因此排序采用分层规则，而不是只用单一 score：

1. `can_fit=true` 的候选优先于 `can_fit=false` 的候选。
2. 同为可容纳或同为不可容纳时，`score` 高者优先。
3. `score` 相近时，`largest_free_region` 大者优先。
4. 仍无法区分时，`free_ratio` 高者优先。

5. 最后使用稳定 tie-break，避免同一输入下排序非确定。

Listing 1: fragmentation_aware 候选排序伪代码

```
for candidate in sampled_segments:
    total_free = sum_free_bytes(candidate)
    total_capacity = sum_capacity(candidate)
    largest = largest_contiguous_free_region(candidate)
    free_ratio = total_free / total_capacity
    contiguity = largest / total_free
    can_fit = largest >= request_size
    score = 0.70 * contiguity + 0.30 * free_ratio

sort candidates by:
    can_fit desc,
    score desc,
    largest desc,
    free_ratio desc,
    stable_index asc
```

3.5 权重选择的工程解释

评分公式中 `contiguity_ratio` 权重为 0.70，`free_ratio` 权重为 0.30。这个选择不是为了追求复杂模型，而是为了表达一个工程优先级：在碎片化问题上，连续空间形态应当成为主要信号，但总空闲比例仍然不能被完全丢弃。如果只按最大连续空闲区域排序，策略可能长期偏向少数拥有大连续块的 `segment`，导致整体空间负载不均衡；如果只按总空闲比例排序，则会回到 `free_ratio_first` 的误判问题。0.70 和 0.30 的组合让策略在两者之间形成明确偏好：先保证当前请求可放入，再兼顾空间均衡。

更重要的是，`fragmentation_aware` 并不直接用 `score` 替代所有排序条件。它先比较 `can_fit`，再比较 `score`。这意味着当候选 A 无法容纳当前请求、候选 B 可以容纳当前请求时，即使 A 的总空闲比例更高，也不会排在 B 前面。这个设计避免了“高分但不可用”的情况。`score` 只在同一 `fit` 状态内部发挥作用，用于区分多个都可用或多个都不可用的候选。

3.6 候选排序示例

为了更直观说明策略行为，可以考虑三个候选 `segment`。Segment A 总空闲最高，但最大连续块不足；Segment B 总空闲中等，最大连续块刚好满足请求；Segment C 最大连续块更大，但总空闲比例较低。若当前请求为 10MiB，则 A 虽然总空闲高，但 `can_fit=false`，会排在 B 和 C 之后；B 和 C 都能容纳请求，再根据综合 `score` 和最大

表 11: 不同排序信号的取舍

排序方式	优点	缺点
只看总空闲比例	有利于 segment 整体空间均衡	无法判断当前请求是否存在足够连续空间
只看最大连续空闲区域	对大对象分配直接有效	可能牺牲整体空间均衡，过度偏向少数 segment
只看连续性比例	能识别空闲空间是否集中	可能忽略绝对空闲容量，较小 segment 也可能得分高
先看 <code>can_fit</code> 再看综合 <code>score</code>	当前可分配性优先，同时保留空间均衡信号	比 <code>free_ratio_first</code> 多一次碎片元数据读取和排序比较

连续区域排序。这样排序结果反映的是“先避免明显失败，再在可用候选中做质量选择”。

表 12: 候选排序示例

segment	总空闲	最大连续空闲	请求大小	<code>can_fit</code>	排序解释
A	20MiB	8MiB	10MiB	false	总空闲最高但不可容纳，排在可容纳候选之后
B	14MiB	10MiB	10MiB	true	可容纳且空闲较均衡，是优先候选之一
C	12MiB	12MiB	10MiB	true	可容纳且连续块更大，与 B 按 <code>score</code> 和 <code>tie-break</code> 比较

3.7 完整策略流程

图3展示了 `fragmentation_aware` 的完整流程。这里需要强调两点。第一，`preferred segment` 阶段不进入评分排序，而是沿用原有优先尝试语义；这是为了保证调用方显式指定的位置仍具有最高优先级。第二，评分排序只发生在采样候选上，且排序后仍逐个调用原有 `allocator` 尝试分配；如果候选不足以满足副本数，策略继续进入原有 `fallback` 路径。

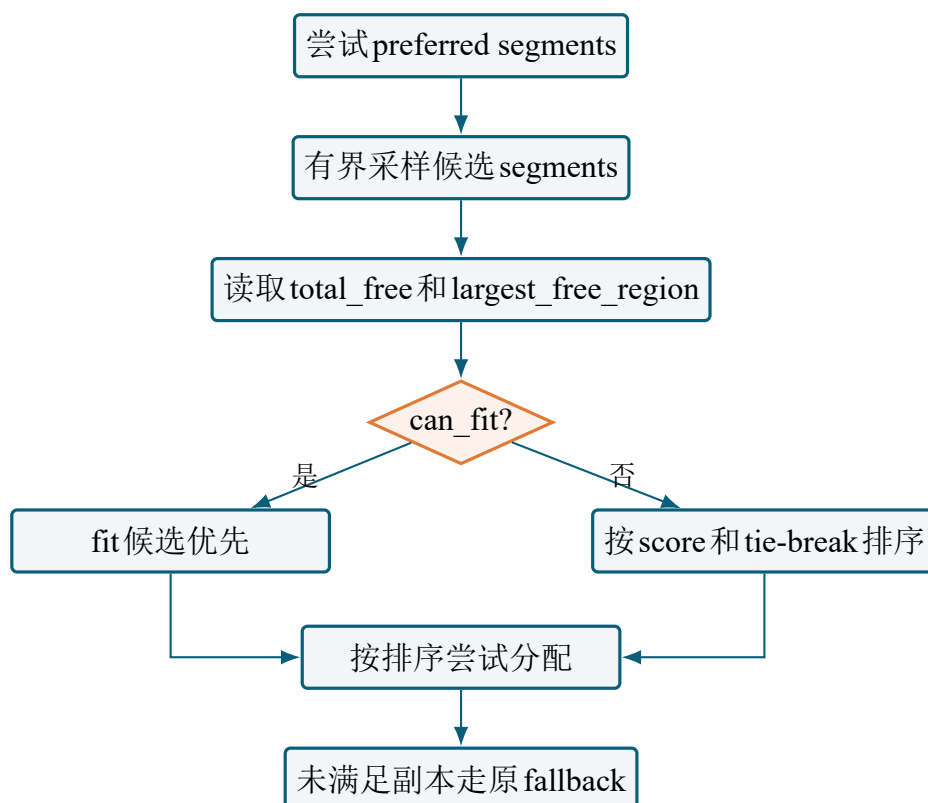


图 3: fragmentation_aware 策略流程

3.8 多副本场景下的处理

Mooncake Store 并不只处理单副本写入。多副本场景下，策略需要同时满足“每个副本找到可用空间”和“同一次请求尽量不要重复使用同一 segment”两个条件。若只为第一个副本选择最优 segment，而忽略后续副本，可能导致后续副本候选空间不足。因此，fragmentation_aware 在每次成功分配后都会将该 segment 加入 used 集合。后续副本采样和分配时会跳过 used segment，从而保留已有策略中的副本去重语义。

这种处理方式也解释了为什么策略没有输出一性全局排序计划。Store 中的空间状态可能在并发请求之间快速变化，提前构造完整计划并不能保证全部成功。更稳妥的方式是：每个副本在当前状态下选择高质量候选，成功后立即更新 used 集合；失败则继续尝试候选或 fallback。这样可以在不引入复杂全局锁和事务计划的情况下，保持较好的工程鲁棒性。

3.9 退化行为与边界保护

一个可合入上游的策略必须考虑不完整信息。并不是所有 allocator 都一定能提供精确的碎片信息；即使能够提供，评分后到实际分配之间也可能发生并发状态变化。因此，fragmentation_aware 从一开始就把碎片信息定位为“排序提示”，而不是“成功承诺”。当 getLargestFreeRegion() 不可用或返回未知值时，策略使用可用空闲

表 13: 多副本分配中的状态变化

阶段	动作	目的
副本开始	读取 <code>excluded</code> 和 <code>used</code> 集合	排除不可选和已使用 <code>segment</code>
preferred 尝试	优先尝试用户指定位置	保留上层调度语义
候选排序	对未排除、未使用候选进行碎片感知排序	提高当前副本首选成功率
分配成功	将 <code>segment</code> 加入 <code>used</code> 集合	避免后续副本重复选择同一位置
分配不足	进入原有 <code>fallback</code> 路径	保证 <code>best-effort</code> 语义不被削弱

空间进行保守估计；当排序后实际分配失败时，策略不会强行占用空间，而是继续尝试后续候选或 `fallback`。

这种设计有两个好处。第一，它避免新策略对所有 `allocator` 形成硬依赖，降低引入成本。第二，它把正确性仍然放在 `allocator` 最终检查上，避免因元数据读取不精确导致空间一致性问题。换句话说，`fragmentation_aware` 优化的是“尝试顺序”，不是“容量判定权”。容量判定权仍然属于底层 `allocator`。

3.10 复杂度与并发安全分析

`fragmentation_aware` 没有引入新的全局数据结构，也没有在策略内部缓存跨请求状态。它只在每次分配时对采样候选读取 `allocator` 元数据，并对候选数组排序。若候选数量为 k ，则额外计算复杂度为 $O(k)$ ，排序复杂度为 $O(k \log k)$ 。由于 k 来自现有有界采样，而不是全量 `segment` 数 N ，策略开销不会随集群规模线性扫描所有 `segment`。

并发方面，新策略不改变 `allocator` 的实际申请路径，也不绕过已有锁和状态检查。评分阶段读取的空闲信息只能作为排序依据，不能作为最终成功承诺；真正分配仍由 `allocator` 执行。因此，即使评分后其他线程改变了某个 `segment` 的空闲状态，最坏结果也只是该候选分配失败并继续尝试后续候选，不会破坏空间一致性。

4 实现路径

表 14: 策略复杂度与风险控制

维度	处理方式	风险控制
候选规模	沿用有界采样	避免全局扫描导致扩展性退化
元数据读取	读取总空闲和最大连续区域	只用于排序，最终结果由 allocator 确认
排序成本	对候选集合排序	候选数量小，本地仿真额外开销约 17.93ns
并发变化	分配前后状态可能变化	保留原有失败重试和 fallback 路径
兼容性	无精确碎片信息时退化处理	不强制改造所有 allocator

4.1 代码改动总览

当前 PR 补丁涉及 9 个文件，覆盖策略实现、类型系统、配置解析、启动参数、测试、benchmark、文档和 CI 辅助步骤。表 15 列出主要改动。

4.2 策略类接入

核心实现位于 `mooncake-store/include/allocation_strategy.h`。新类 `FragmentationAwareAllocationStrategy` 继承现有随机策略基础能力，而不是重写所有分配流程。这样做有两个好处：第一，可以复用原有单 segment 分配、随机 fallback 和排除集合处理；第二，可以把新增逻辑限制在候选评分和排序层，降低对主路径的侵入。

实现上，新策略先按原有语义处理 preferred segment。如果 preferred segment 没有被排除、没有被当前请求重复使用，并且 allocator 能够完成申请，则直接返回该分配结果。只有在 preferred 阶段无法满足全部副本时，策略才进入候选采样和碎片感知排序。排序后的候选按顺序尝试分配，成功后加入 used set，避免同一次多副本请求重复选择同一 segment。

4.3 核心执行流程

从代码路径看，`fragmentation_aware` 的核心执行可以分为五个阶段。第一阶段是预处理，把 excluded segment 和当前请求中已经成功使用的 segment 作为不可选集合。第二阶段是 preferred 尝试，按调用方传入顺序处理 preferred segment。第三阶段是候选

表 15: 关键代码改动

文件	作用
mooncake-store/include allocation_strategy.h	新增 <code>FragmentationAwareAllocationStrategy</code> , 实现候选评分、排序、 <code>preferred</code> 处理和 <code>fallback</code> 衔接。
mooncake-store/include types.h	新增 <code>FRAGMENTATION_AWARE</code> 枚举值, 让策略进入统一类型系统。
mooncake-store/include master_config.h	解析 <code>allocation_strategy=fragmentation_aware</code> , 非法值继续走原有报错路径。
mooncake-store/src master.cpp	更新启动参数帮助信息, 确保用户可以从命令行发现新策略。
mooncake-store/tests allocation_strategy_test.cpp	增加碎片化定向测试、 <code>preferred segment</code> 测试和参数化覆盖。
mooncake-store benchmarks allocation_strategy_bench.cpp	将新策略加入分配策略 <code>benchmark</code> 矩阵, 便于后续真实环境对比。
docs/source/design mooncake-store.md	增加策略说明、适用场景和与其他策略的选择建议。
docs/source/deployment mooncake-store-deployment -guide.md	增加部署参数说明, 给出启用方式。
.github/workflows ci.yml	在 Go store binding 集成测试前释放 runner 磁盘空间, 避免无关环境失败。

采样，沿用已有策略体系的有界采样思想，从剩余 `segment` 中得到候选集合。第四阶段是候选评分，对每个候选读取总容量、总空闲、最大连续空闲区域，并计算 `can_fit` 和 `score`。第五阶段是按排序结果尝试实际分配，成功则记录结果，失败则继续尝试后续候选；如果仍不能满足副本数，则回到原有 `fallback` 路径。

Listing 2: `fragmentation_aware` 实现层执行逻辑概览

```
Allocate(manager, slice_length, replica_num, preferred, excluded):
    used = {}
    result = []

    for name in preferred:
        if name in excluded or name in used:
            continue
        buffer = allocateSingle(manager, name, slice_length)
        if buffer.ok:
            result.push(buffer)
            used.insert(name)
        if result.size == replica_num:
            return result

    candidates = sampleCandidateSegments(manager, excluded, used)
    ranked = []
    for name in candidates:
        metrics = collectFragmentationMetrics(manager, name)
        ranked.push(scoreCandidate(name, metrics, slice_length))

    sort(ranked, FragmentationAwareComparator)

    for candidate in ranked:
        if candidate.name in excluded or candidate.name in used:
            continue
        buffer = allocateSingle(manager, candidate.name, slice_length)
        if buffer.ok:
            result.push(buffer)
            used.insert(candidate.name)
        if result.size == replica_num:
            return result

    return randomBestEffortFallback(manager, result, excluded, used)
```

这段逻辑体现了一个重要原则：新策略并没有接管底层空间管理，也没有绕过已有 `allocateSingle` 路径。它只在调用 `allocateSingle` 之前调整候选顺序。这样做

能最大限度降低行为风险，因为空间是否真的可分配仍由原有 `allocator` 确认；同时，新策略又能在排序阶段规避最明显的碎片化误判。

表 16: 实现阶段与代码职责

阶段	代码职责	设计意图
preferred 阶段	遍历 <code>preferred segment</code> 并尝试分配	保持调用方显式放置优先级
采样阶段	从可用 <code>segment</code> 中得到候选集合	保持策略开销有界，避免全局扫描
评分阶段	收集总空闲、容量和最大连续空闲区域	把碎片信息转化为排序字段
排序阶段	按 <code>can_fit</code> 、 <code>score</code> 和 <code>tie-break</code> 排序	让可容纳当前请求的候选优先
分配阶段	调用原有 <code>allocator</code> 申请空间	保证最终正确性仍由 <code>allocator</code> 判定
fallback 阶段	未满足副本时走原有兜底逻辑	保留 <code>best-effort</code> 副本语义

4.4 配置和工厂接入

为了让策略成为正式可选项，而不是测试代码中的孤立类，PR 在类型、配置和工厂三处同时接入：

1. 在 `AllocationStrategyType` 中新增 `FRAGMENTATION_AWARE` 枚举值。
2. 在 `master_config.h` 中识别字符串 `fragmentation_aware`。
3. 在策略工厂中为 `FRAGMENTATION_AWARE` 返回新策略实例。
4. 在 `master.cpp` 和部署文档中更新 `--allocation_strategy` 可选值。

新策略通过 Master 启动参数显式启用：

```
mooncake_master --allocation_strategy=fragmentation_aware
```

由于默认策略保持为 `random`，现有部署不会因为合入该 PR 而改变行为。若线上灰度时发现新策略不适合某类负载，只需要切回 `random` 或 `free_ratio_first` 即可回滚。

4.5 兼容性、灰度与回滚

开源项目中的性能优化如果默认改变行为，评审风险会明显增大。Mooncake Store 面向真实推理服务，用户对默认策略的吞吐、稳定性和资源使用已经形成预期。因此，本项目把 `fragmentation_aware` 设计为显式启用策略，而不是替换默认策略。这样既可以让需要碎片优化的用户主动选择，也避免对现有部署产生隐式影响。

灰度使用时，可以先在测试环境或单个 Store 实例中启用新策略，观察失败分配次数、fallback 次数、写入延迟和空间利用率。如果指标改善，再扩大到更多实例；如果发现某类负载下新策略收益不足或元数据读取成本偏高，只需要调整启动参数即可回滚，不需要回滚二进制或修改客户端。

表 17: 策略启用和回滚方式

场景	配置	说明
默认行为	不传或使用 <code>random</code>	完全保留 Mooncake Store 现有默认策略
空间均衡基线	<code>--allocation_strategy=free_ratio_first</code>	使用总空闲比例优先策略，作为对照基线
碎片优化	<code>--allocation_strategy=fragmentation_aware</code>	启用本项目新增策略，适合混合大小对象和长期运行场景
快速回滚	改回 <code>random</code> 或 <code>free_ratio_first</code>	不需要修改客户端协议，不需要迁移已有数据

4.6 单元测试设计

测试不是简单地检查“新策略能运行”，而是针对问题定义构造确定性场景。核心测试创建两个 segment: `fragmented` 和 `contiguous`。前者总空闲空间更高，但最大连续区域小于请求大小；后者总空闲空间较低，但最大连续区域可以容纳请求。断言目标是: `fragmentation_aware` 应该选择 `contiguous`，而不是被更高总空闲比例误导。

另一个关键测试是 `preferred segment` 语义。即使某个 `preferred segment` 在评分上不是最优，只要它没有被排除且可以满足请求，新策略仍应优先选择它。这一点对兼容性很重要，因为 `preferred segment` 通常来自上层调度或副本放置约束，不能被内部评分逻辑无条件覆盖。

碎片化测试的构造要避免一个常见陷阱：如果释放的是靠近尾部的块，`allocator` 可能把释放块和尾部剩余空间合并，导致最大连续空闲区域大于预期。这样测试看似构造了碎片，实际却没有形成“大量空闲但连续块不足”的状态。最终 PR 中的测试选择

释放内部块，确保 **fragmented** 的总空闲空间较高，但最大连续空闲区域仍小于请求大小；同时构造 **contiguous**，使其总空闲空间较低但最大连续区域满足请求。这种测试更接近问题定义，能稳定验证排序信号是否正确。

表 18: 碎片化单元测试的构造逻辑

步骤	构造动作	验证意义
构造 fragmented	分配多个固定大小块，再释放内部块	形成总空闲较高但连续空闲不足的 segment
构造 contiguous	保留一段足够大的连续空闲区域	提供能够满足当前请求的对照 segment
设置请求大小	请求大于 fragmented 最大连续块，小于 contiguous 最大连续块	强制暴露总空闲比例误判问题
执行新策略	调用 fragmentation_aware 进行分配	验证策略优先选择可容纳请求的 segment
检查 preferred	单独传入 preferred segment	验证兼容性语义没有被评分逻辑破坏

4.7 测试修复与CI闭环

PR 创建后经历了三类 CI 问题并逐步修复。第一次失败来自代码格式检查，后续通过格式化修复。第二次失败来自 GitHub runner 磁盘不足，Go store binding 集成测试在链接阶段出现空间不足；这不是策略逻辑失败，但会阻断 PR 验证，因此在对应 job 前增加了磁盘清理步骤。第三次失败来自最初碎片化测试构造不够严格：释放块与尾部空闲区域发生合并，导致最大连续区域大于预期，断言不稳定。后续测试改为构造 5 个 8MiB 块并释放内部块，避免尾部合并影响碎片形态。

4.8 文档、benchmark 与复现材料

除了代码本身，PR 和比赛材料还补充了三类工程化内容。第一，在 Mooncake 官方文档中说明 **fragmentation_aware** 的适用场景和启用方式，避免评审只看到代码而不知道何时使用。第二，将新策略加入 **allocation_strategy_bench.cpp**，让后续真实 Store benchmark 可以直接横向比较 **random**、**free_ratio_first** 和 **fragmentation_aware**。第三，在比赛材料仓库中提供独立复现实验和日志，用轻量

表 19: 关键测试覆盖点

测试点	构造方式	验证目的
碎片化选择	构造高总空闲但最大连续区域不足的 segment	验证策略不会只按总空闲比例排序
连续空间优先	构造总空闲略低但连续空间足够的 segment	验证 <code>can_fit</code> 优先级生效
preferred 保留	传入可用 preferred segment	验证用户显式优先级不被评分覆盖
参数化覆盖	将新策略加入既有策略测试矩阵	验证基础分配语义与已有策略一致
benchmark 接入	加入分配策略 benchmark 列表	为后续真实性能对比提供入口

表 20: CI 问题定位与处理

问题	原因	处理结果
格式检查失败	新增 C++ 代码未完全符合项目格式	运行格式化并重新提交
runner 磁盘不足	GitHub Actions 环境空间不足, 影响 Go store binding 测试	增加 CI 磁盘清理步骤, 避免无关失败
碎片化测试失败	测试构造中空闲块与尾部空闲区域合并	调整为内部块释放构造, 保证最大连续区域低于请求大小
最终验证	上述问题修复后重新触发 CI	PR 头提交 <code>0123fa1</code> 通过 26 个检查, 1 个检查跳过

仿真固定问题模型，证明改进来自“排序信号变化”，而不是扩大候选规模或修改实验口径。

5 验证与效果

5.1 验证思路

本项目的验证分为三层。第一层是代码级验证，确认新策略能够编译、能够被配置解析、能够进入策略工厂，并且不会破坏已有分配策略测试。第二层是问题级验证，通过最小碎片化测试证明策略确实解决“总空闲比例高但最大连续空闲不足”的误判。第三层是专题仿真验证，通过多个请求和多个segment构造，观察首选成功次数、fallback尝试次数和单次决策成本。三层验证分别回答“能不能运行”“是否解决目标问题”“收益和成本如何”。

需要说明的是，初赛阶段的验证重点是策略层正确性和可复现性，而不是宣称已经完成完整生产压测。真实RDMA环境、真实SGLang HiCache端到端吞吐和长时间集群稳定性测试需要更多硬件和部署条件，适合作为决赛阶段继续推进。本报告中的数据口径均围绕策略层展开，不把离线仿真结果夸大为生产集群吞吐结论。

表 21: 验证层次与回答的问题

验证层次	主要材料	回答的问题
代码级验证	单元测试、参数化测试、上游CI	新策略是否能安全接入Mooncake代码路径
问题级验证	碎片化定向测试	新策略是否真的避免总空闲比例误判
指标级验证	专题仿真和日志	首选成功率、fallback次数和决策开销如何变化
工程级验证	文档、benchmark接入和PR流程	后续评审者是否能复现、理解并继续扩展

5.2 CI验证

Mooncake上游draft PR状态如表22所示。

表 22: PR CI 状态

项目	内容
PR 链接	https://github.com/kvcache-ai/Mooncake/pull/2797
头提交	0123fa1 Fix fragmentation-aware allocation test setup
CI 结果	26 个检查成功, 1 个检查跳过
结论	上游构建、格式检查、文档构建和相关测试已通过。

5.3 专题仿真指标

专题仿真使用 5 个 segment 和 6 个大对象请求, 模拟高总空闲但碎片化、低总空闲但连续空间充足两类 segment 并存的 Store 状态。结果如表 23 所示。

这里的“首选 segment 可直接容纳”指排序后的第一个候选是否能够立即满足当前请求。这个指标非常关键, 因为它衡量的是策略排序质量, 而不是系统最终是否能够通过 fallback 补救。“最终可容纳”表示在继续尝试其他候选后, 请求最终是否能找到空间; 它用于确认两个策略面对同一容量条件时并没有改变总可用容量。“fallback 尝试次数”则用于衡量首选错误带来的额外路径压力。平均候选 segment 数量用于证明新策略没有通过扩大搜索范围获得收益, 而是在相同候选规模内改进排序。

表 23: 专题对齐仿真结果

指标	free_ratio_first	fragmentation_aware
大对象首选 segment 可直接容纳	0/6	6/6
最终可容纳	6/6	6/6
fallback 尝试次数	11	0
平均候选 segment 数量	5.00	5.00
单次决策耗时	121.00ns	138.93ns
新增决策开销	不适用	17.93ns

5.4 结果解释

该结果有三个含义。第一, fragmentation_aware 显著改善首选 segment 质量, 在所有测试请求上都能首先选中可容纳 segment。第二, 平均候选 segment 数量没有增加, 说明改进来自排序信号变化, 而不是扩大搜索范围。第三, 单次决策新增开销约 17.93ns,

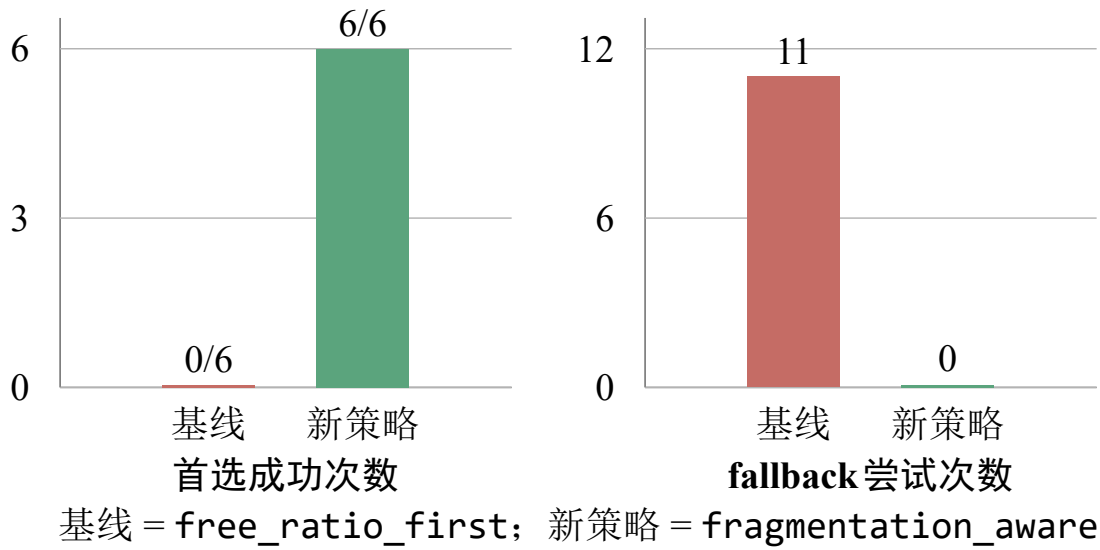


图 4: 首选成功次数与 fallback 尝试次数对比

在本地 CPU 仿真中处于很小量级，符合“用少量元数据计算换取更稳定分配路径”的设计目标。

进一步看，free_ratio_first 最终也能达到 6/6 可容纳，说明测试场景中系统总容量并不缺失；问题在于它先尝试了不合适的 segment，需要额外 fallback。换言之，本项目优化的不是“凭空增加容量”，而是“更快找到已经存在的正确位置”。这也是 Store 分配策略优化的现实价值：在容量足够但碎片化严重的状态下，吞吐和尾延迟问题往往来自无效尝试、重试和锁路径放大，而不一定来自物理资源绝对不足。

单次决策耗时从 121.00ns 增加到 138.93ns，新增约 17.93ns。这个成本主要来自读取最大连续空闲区域、计算连续性比例和增加排序比较字段。与一次真实网络 I/O、远程存储访问或复杂推理请求相比，这一级别开销很小；但报告仍然保留该成本，而不把策略描述为“零开销”。更准确的结论是：fragmentation_aware 用可控的纳秒级决策开销，换取碎片化场景下明显更高的首选成功率和更少 fallback 尝试。

6 与赛题要求的对应

表 24: 赛题要求映射

赛题关注点	本项目实现	当前证据
Mooncake Store 吞吐性能	减少碎片化导致的无效首选分配和 fallback 压力	专题仿真中 fallback 尝试次数从 11 降为 0

可扩展性	保持有界候选采样，不做全局扫描	平均候选 segment 数量保持 5.00
高可用语义	不改变副本 best-effort 路径，保留 preferred/excluded 语义	单元测试和策略设计说明
SGLang HiCache+Store	优化 Store 后端对象放置路径，为 HiCache 后端稳定性提供基础	方案定位和代码路径位于 Mooncake Store
开源规范	以 PR 形式提交上游 Mooncake，并保留 patch 和文档	PR#2797 通过 CI
可复现性	提供仿真源码、日志、技术报告和使用说明	repro/、logs/、EVALUATION.md

7 使用与复现

7.1 启用策略

```
mooncake_master --allocation_strategy=fragmentation_aware
```

7.2 应用补丁

```
git apply --check mooncake_fragmentation_aware_pr_2797_0123fa1.patch
git apply mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

7.3 回滚补丁

```
git apply -R mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

7.4 复现实验

```
g++ -std=c++17 -O2 repro/fragmentation_aware_sim.cpp -o
    fragmentation_aware_sim
./fragmentation_aware_sim
```

```
g++ -std=c++17 -O2 repro/fragmentation_aware_metrics.cpp -o
    fragmentation_aware_metrics
./fragmentation_aware_metrics

g++ -std=c++17 -O2 repro/topic_aligned_store_scalability_sim.cpp -o
    topic_aligned_store_scalability_sim
./topic_aligned_store_scalability_sim
```

8 边界与后续工作

本项目当前阶段已经完成代码实现、上游PR、CI验证、仿真评估和中文提交材料整理，但仍有边界需要明确：

- 当前不宣称已经完成真实RDMA硬件benchmark。
- 当前不宣称已经完成真实SGLang HiCache端到端吞吐压测。
- 当前不宣称已经重设计Mooncake高可用协议。
- 当前专题仿真验证的是分配策略层，不等同于完整生产集群压测。

后续如果进入决赛，建议补充三类工作：第一，在真实Mooncake Store环境中运行allocation_strategy_bench，统计吞吐、P50/P99、失败分配次数、fallback次数和内存利用率；第二，在具备RDMA条件的集群中验证跨节点场景；第三，将SGLang HiCache接入Mooncake Store后端，使用真实推理负载验证KVCache复用和后端IO表现。

9 结论

本文围绕Mooncake赛题2提出并实现Mooncake Store碎片感知分配策略。该策略针对free_ratio_first只看总空闲比例的不足，引入最大连续空闲区域和可容纳性排序，在不改变默认行为、不扩大候选规模、不破坏原有副本语义的前提下，提升碎片化场景下的大对象首选分配质量。当前PR已经通过上游GitHub Actions，确定性仿真显示首选成功次数从0/6提升到6/6，fallback尝试次数从11降到0。该工作是一个边界清晰、可复现、可回滚、适合继续扩展到真实Store benchmark和SGLang HiCache端到端测试的初赛成果。

A 提交材料清单

文件或目录	说明
README.md	项目入口说明
SUBMISSION.md	平台提交说明
DESIGN.md	设计文档
technical_solution.md	技术方案说明
EVALUATION.md	评估报告
testing.md	测试说明
usage.md	使用说明
REVIEW_GUIDE.md	评审阅读指南
mooncake_fragmentation_aware pr_2797_0123fa1.patch	当前PR补丁
repro/	复现实验源码
logs/	验证日志
slides/	初赛展示PPT
report/	本技术报告源码和PDF
release/	提交压缩包和校验文件

B 参考链接

- Mooncake 上游仓库: [kvcache-ai/Mooncake](#)
- 本项目 Mooncake PR: [kvcache-ai/Mooncake#2797](#)
- 比赛材料仓库: [Lorry1024/ccf2026-mooncake-fragmentation-aware](#)
- Mooncake fork 分支: [Lorry1024/Mooncake:ccf-fragmentation-aware-allocation](#)